

Vericoding: Specification-Centric Software Development for the AI Era

How cheap implementation makes specifications the primary artifact

Gavin Mendel-Gleason
Scidonia

Abstract

Large language models have sharply reduced the cost of producing implementations but have not reduced the cost of deciding whether an implementation satisfies intent. Mainstream software engineering still treats source code as the primary maintained artifact, with specifications, tests, and proofs as secondary validation. We argue that the changed economics invert this relationship. When implementation is cheap and verification stays expensive, the scarce resource is *specification*, and the durable artifact should be the specification rather than the code. We call the resulting model *vericoding*: developers iteratively refine a strong specification, an AI system synthesizes the implementation and the subsidiary specifications its proof needs, and a verifier returns a deterministic verdict – with a structured residual, not a vague complaint, on failure. We make the case for this model, identify the properties it demands of a verifier (compositional reasoning so that proofs behave like cached build artifacts, and structured failure so that an AI can repair against a proof obligation rather than re-guess against a prompt), and describe its realization in Axiomander, a working verifier for Python whose contracts are ordinary `assert` statements. We give an evaluation methodology built around retrofitting, synthesis, and maintenance, report early evidence from the prototype, and discuss the human-factors and soundness questions the model raises. The central claim is economic rather than technical: software development in the AI era is better understood as specification management than as programming.

Keywords

vericoding, specification-centric development, deductive verification, LLM code synthesis, contracts, compositional verification, separation logic, software engineering process

1 Introduction

For most of the history of software engineering, writing code was the expensive step. Processes, tools, and review practices grew up around that assumption: the source code is the artifact a team reads, edits, owns, and trusts, and tests sample its behaviour from the side. Specifications, where they exist at all, are usually informal.

Large language models change the cost structure. Generating a plausible implementation of a described task is now fast and cheap. Deciding whether that implementation is *correct* is not: it is exactly as hard as it ever was, and arguably harder, because an LLM is trained to satisfy the *prompt*, not the *intent*. Its characteristic failure is silent – code that compiles, runs on the example, and reads correctly to a skimming reviewer, while quietly mishandling the inputs nobody thought about. The bottleneck has moved from implementation effort to intent specification.

This paper takes that shift seriously and asks what development looks like when specification, not code, is the scarce and therefore primary artifact. We call the model *vericoding*:

Traditional: Human $\rightarrow$ Code $\rightarrow$ Tests $\rightarrow$ Confidence

Vericoding: Human $\rightarrow$ Specification $\rightarrow$ AI $\rightarrow$ Implementation $\rightarrow$ Verification

The human writes and maintains a strong specification. An AI derives an implementation – and, freely, the helper functions and *their* specifications that its proof needs. A verifier decides, deterministically, whether the code meets the specification, and on failure hands back a structured residual that drives the next attempt. The implementation becomes a derived, disposable artifact; the specification persists across many implementations.

Two properties make this loop tractable rather than aspirational, and they are the engineering content of the paper. First, **compositional verification**: a caller must be provable from a callee’s contract, not its body, so that editing a body does not re-verify the world. Under this discipline a proof behaves like a cached object file and the verifier behaves like a build system for correctness. Second, **structured failure**: a negative verdict must return the exact remaining obligation (with hypotheses, and a counterexample where one exists), so that the AI repairs against a goal rather than re-guessing against a prompt.

Contributions.

- We articulate vericoding as a software-engineering model and state its four operating principles (Section 3), framing the shift as an economic consequence of cheap implementation.
- We argue that the model stands or falls on *compositional scaling*, and show how contracts-as-interfaces plus proof caching turn verification into an incremental build system for correctness (Section 6).
- We describe the *heal loop* – residual-driven repair – and why it dominates naive re-prompting (Section 7).
- We report a concrete realization, Axiomander, a verifier for Python whose contracts are plain `assert` statements and whose verdict bottoms out in a machine-checked proof (Section 5), and give an evaluation methodology with early evidence (Section 8).
- We surface the human-factors and soundness questions the model raises (Sections 9–10).

2 The traditional model and its limits

Traditional development is code-centric: the human writes code, tests sample it, and confidence accumulates through review. This made sense when humans were the code generators – the artifact you paid to produce was the artifact you kept. The weakness

Table 1: A ladder of guarantees. Each mechanism catches more, but only a strong specification states intent totally.

Mechanism	Catches	Misses
Prompt review	gross misunderstanding	everything subtle
Types	shape errors	values, relations, invariants
Tests	cases you thought of	cases you did not
Strong spec.	anything expressible	only what you leave unstated

is in what the validation mechanisms actually guarantee. Types constrain the *shape* of data but say nothing about the relationship between inputs and outputs; tests assert facts at finitely many points. Both are *samples* of intent, not statements of it.

LLM-generated code stresses exactly this gap (Table 1). The model is not adversarial; it is *optimistic*. It assumes the inputs it did not consider behave like the ones it did, that an unwritten invariant holds, and that an omitted error path is unreachable. A green test suite says “these inputs are right”; it cannot say “every input is right.” As implementation volume rises and human reading time does not, sampling-based validation falls further behind. What is needed is a mechanism that decides correctness across *all* inputs, deterministically – which is what a proved specification provides.

3 The vericoding model

Vericoding inverts the data-flow of development. Rather than writing code and sampling it, the developer writes a specification and derives code beneath it:

$$\underbrace{(P, Q)}_{\text{spec, human-authored}} \rightsquigarrow \underbrace{c}_{\text{code, AI-derived}} \quad \text{such that} \quad \models \{P\} c \{Q\}.$$

The human authors both halves: the assumption P on inputs and the guarantee Q on outputs. The code is derived so that Q holds under P . Four principles follow.

Principle 1: specification is the control surface. You steer the system by editing specifications, not code. The reviewable surface shrinks from “all the code” to “the contracts at the boundary.”

Principle 2: code is a derived artifact. The implementation is disposable. If it does not verify, regenerate it. We do not trust the code because we read it; we trust it because it was *proved* against a specification we did read.

Principle 3: correctness requires proof. A specification is only as good as our ability to discharge it. Runtime contracts catch violations after deployment on whatever inputs occur; static proof catches them before deployment on all inputs. The verdict is binary and deterministic.

Principle 4: failure returns structured information. A verifier that says only “no” is useless to an automated repair loop. The verdict must carry the residual obligation and, where the goal is false, a concrete counterexample.

These principles only pay off if two scaling problems are solved – composition (Section 6) and structured repair (Section 7) – and

if a real verifier can implement them over a mainstream language (Section 5).

4 Specification management

If code is derived, the central human activity is no longer editing implementations but *managing specifications*: refining contracts, reviewing them, decomposing them, and evolving requirements. This shifts effort toward artifacts that are smaller, more declarative, and easier to review than the code they constrain.

A key degree of freedom belongs to the AI, not the human. To prove a top-level entry point, the AI may decompose the problem into helper functions, each with its *own* contract; those sub-specifications are how it makes the global proof go through, and they are checked the same way – there is no privileged, unverified layer. The human owns the boundary contract; the AI owns the interior decomposition. Formally, a caller’s proof uses only a callee’s contract:

$$\frac{\{P_g\} g \{Q_g\} \quad \{P_f\} f \{Q_f\} \text{ using } Q_g}{\{P_f\} f \{Q_f\}}$$

The body of g never appears in f ’s proof. This is what makes the human’s review surface small even when the generated interior is large, and it is the same property that makes the verifier incremental (Section 6).

5 Realization: Axiomander

Vericoding is a methodology, not a tool, but a methodology is only credible if it runs. Axiomander is a working realization for Python. Its trust kernel is the Rocq proof assistant (formerly Coq); separation logic, via the Iris framework, supplies the weakest-precondition calculus and the heap reasoning that Python’s mutation and aliasing demand; and an LLM acts as an oracle *inside* the loop, never as the trust kernel. The methodology is backend-agnostic – any engine that returns a sound, deterministic decision plus a structured residual would serve – but the realization commits to Rocq+Iris because separation logic is the right foundation for resource ownership in real code.

Contracts are ordinary Python. The user writes no imports and no decorators. Contracts are plain `assert` statements, classified by position, or a verifier-only axiomander: `docstring`:

```
def clamp(val: int, lo: int, hi: int) -> int:
    assert lo <= hi                # precondition
    if val < lo: result = lo
    elif val > hi: result = hi
    else: result = val
    assert lo <= result <= hi      # postcondition
    assert implies(val < lo, result == lo)
    assert implies(val > hi, result == hi)
    return result
```

Leading assertions are preconditions, trailing assertions after the result are postconditions, and loop-body assertions are invariants. A shared linter lowers both syntaxes to one contract IR, so the user’s code stays dependency-free while the verifier does the work.

From Python to a verdict. The pipeline lowers the contract IR to an intermediate imperative language, emits a weakest-precondition obligation, and discharges it through a ladder that runs cheap,

Table 2: Axiomander’s proof pipeline. Deterministic tactics run first; the LLM is the fallback, not the first resort.

Tier	Mechanism	Handles
1	deterministic Rocq (wp_reduce, lia)	assignments, branches, loops, frames
2	SMT / Rocq-hammer	linear/nonlinear arithmetic, FOL
2b	theory-SMT (QF_SLIA)	strings, regex, float dimensions
3	LLM oracle	invariants, induction, lemma chains

Table 3: Incremental invalidation. The unit of reuse is the contract, not the file.

Change	Re-verify
body only	the function
local assert / invariant	the function
contract	the function + transitive callers
callee contract (summary hash)	the caller

deterministic tactics first and escalates only on residuals (Table 2). The defining theorem – that a Hoare triple holds iff the precondition implies the weakest precondition of the body – is proved sound in Rocq/Iris, so an accepted verdict is trustworthy by construction, and the LLM, which may be wrong, only ever proposes proofs that Rocq re-checks.

6 Compositional scaling: a build system for correctness

Vericoding lives or dies on composition. If a one-line spec edit re-verified the whole program, the loop would not scale to real codebases. The discipline, borrowed from incremental build systems, is:

Body changes invalidate local proofs; contract changes invalidate callers.

A contract is an interface; a proof is a cached build artifact. Each function exports a semantic summary

$\text{summary}(g) = \text{hash}(\text{pre}, \text{post}, \text{reads}, \text{writes}, \text{raises}),$

and a caller f depends on $\text{summary}(g)$, not on g ’s implementation. If g ’s body changes but its summary is stable, f ’s proof is reused; if the summary changes, only f and its transitive callers are re-verified (Table 3). The verifier becomes an incremental build system for correctness, not a batch theorem prover, and the same opaque-contract boundary serves three purposes at once: modular verification, proof reuse under iteration, and *stubbing* – an unverified library is replaced by its declared contract, which becomes an axiom available to callers, with the trust boundary made explicit and reviewed once.

Composition needs the right logic. The frame property – reasoning about a component is insensitive to state it does not touch – is what makes proofs local. Whether it is cheap depends on the assertion logic. In a companion study we built the same backend twice: once over a flat name-to-value store, where locality has to be *reconstructed* by hand, and once over Iris separation logic, where it is the native shape of the logic. The difference was stark and is

worth stating concretely, because it is the difference between vericoding scaling and not. A two-call function (two increments and a sum) required, over the flat store, two explicit frame applications plus fourteen variable-preservation conjuncts threaded through intermediate postconditions and a 29-line, hand-decomposed proof that the monolithic tactic could not close; the separation-logic backend discharged the analogous chain with *zero* framing obligations in an 11-line proof that the generator emitted and the kernel re-checked. The gap is not only efficiency. Functions that mutate an owned field, or mutate state and then raise through it, have *no* flat-store rendering at all – the flat store cannot express ownership of a mutable cell. The substructural logic is what makes such programs *verifiable in the first place*, and realistic Python is full of them. Composition, in other words, is both a scaling property and an expressivity property.

7 The heal loop

Naive AI coding is a guessing loop: prompt, code, “try again,” code. The model gets no signal it can act on. Vericoding replaces this with structured repair:

$\text{spec} \rightarrow \text{code} \rightarrow \text{verify} \rightarrow \text{residual goal} \rightarrow \text{repair} \rightarrow \dots$

Two things distinguish a heal loop from re-prompting. A **residual goal** is the exact remaining obligation with its hypotheses; a **counterexample** is a concrete, typed witness of failure. Consider a contract whose pre- and post-conditions are regular-expression membership:

```
def accept(p: str) -> str:
    """ axiomander:
        requires: p.re_match("[0-9]{3}")
        ensures:  result.re_match("[a-z]+") """
    result = p
    return result
```

The verifier does not say “wrong.” It returns a witness – $\text{result} = \text{"000"}$ satisfies the requires-regex and violates the ensures-regex – because a digit string can never match $[a-z]^+$. The AI repairs against that fact. Crucially, obligations are kept small and stably identified (e.g. `sort.insert.preserves_sorted.branch_2`) rather than fused into one giant theorem, so a failed stage yields a reusable artifact – a named residual with its hypotheses – rather than a dead end, and per-obligation caching, parallelism, and narrow AI prompts all follow from the same structure.

8 Evaluation methodology

Vericoding makes empirical claims – that the maintained surface shrinks, that iteration stays local, that structured repair converges – and they must be measured rather than asserted. We propose three case studies and the metrics each isolates, then report what the current prototype already demonstrates.

Case study 1: retrofitted verification. Add contracts to an existing Python codebase and measure contracts written per function, proof obligations generated, and the fraction discharged at each tier. This isolates the cost of *adopting* the model on code not written for it.

Case study 2: synthesis from specification. Generate implementations from specifications alone and measure repair iterations to

a verified result, per-tier proof-success rates, and residual complexity at the point the AI takes over. This isolates the heal loop’s convergence.

Case study 3: maintenance under changing requirements. Take a requirement change and run it both ways – traditional (edit code, update tests) and vericoding (edit specification, regenerate, re-verify) – measuring engineering effort and the blast radius of re-verification. This isolates the incremental-build claim of Section 6.

Early evidence. The prototype already exercises the mechanisms these studies will quantify, which is what makes the studies runnable rather than hypothetical. Axiomander verifies end-to-end on a corpus of contracted functions; on that corpus the deterministic tier discharges the large majority of goals (roughly four-fifths in our runs) before any external prover is invoked, with SMT and the LLM handling the residual. It returns typed counterexamples for false goals, including the regex case above, and it dogfoots: more than a dozen of Axiomander’s own functions – covering enum dispatch, instance-based case analysis, string methods, and object construction – carry contracts and are verified by the tool itself. The compositional measurement of Section 6 is a controlled comparison on the same functions across two backends. We are explicit that this is prototype-stage evidence, not a completed controlled study; the three case studies above, on third-party code and with human-effort measurement, are the work that turns it into one.

9 Human factors

Vericoding makes a falsifiable bet about where engineers spend attention. The hypothesis is that traditional development asks humans to read large volumes of implementation, whereas vericoding asks them to read comparatively small contract surfaces, and that the durable artifacts (specifications) and the churning artifacts (implementations) separate cleanly. The questions worth measuring are direct: what do engineers actually review, which artifacts stay stable across a feature’s life, which churn, and how much generated code a human must read before trusting a verified result. If, in practice, reviewers feel compelled to read the generated code anyway, the central economic claim weakens; if they review contracts and trust proofs, it holds. These are usability and trust questions, and they are as central to the model’s success as the logic underneath it.

10 Limitations and threats to validity

Vericoding shifts effort; it does not eliminate it. The most important shift is also the most important risk: the specification is now the artifact that can be wrong. A proof against an incorrect specification certifies the wrong thing, and a specification can be vacuous (an unsatisfiable precondition proves anything). Mitigations – sanity-checking preconditions for satisfiability, surfacing vacuity, and reviewing contracts as the primary artifact – reduce but do not remove this risk.

Three further limitations bound the model. *Soundness of assumptions:* SMT-discharged facts imported as axioms, and library stubs taken on faith, are trust boundaries; they must be logged and auditable, and a wrong stub is a silent hole. *Coverage:* a verifier supports a language fragment, and constructs outside it (runtime

metaprogramming, reflection) must be gated rather than silently mistranslated. *Scalability of proof search:* the LLM tier can fail to find an invariant or a lemma chain, in which case the loop stalls and a human must intervene. Finally, our evaluation is at present prototype-stage and self-referential; the controlled case studies of Section 8, on code we did not write, are required before the model’s effort claims can be considered established.

11 Related work

Deductive verification has a long lineage – Hoare logic [1], Dijkstra’s weakest preconditions [2], and separation logic [3] as realized in modern frameworks such as Iris [4]. Auto-active verifiers including Dafny [5], F* [6], and Verus [7] let programmers discharge functional-correctness obligations with substantial automation. Axiomander reuses this machinery; its departure is not in the logic but in the *process context*. Where these systems expose the assertion logic to the programmer and treat the annotated program as the artifact, vericoding keeps the logic an implementation detail (contracts are plain Python), makes the specification the maintained artifact, and places an LLM in a structured repair loop over residual proof state.

On the synthesis side, large language models generate code at scale [8] but optimize for prompt resemblance, not verified intent; the gap they leave is precisely the one a verifier closes. The contribution we claim is a coherent software-engineering model that joins these threads: cheap AI synthesis under a human-maintained specification, made to scale by compositional, contract-keyed proof caching, and made convergent by residual-driven repair.

12 Conclusion

The observation behind vericoding is economic, not technical. When implementation becomes inexpensive and verification stays expensive, the specification becomes the scarce resource and, naturally, the primary maintained artifact. The engineering consequences are concrete: the reviewable surface shrinks to contracts, proofs become cached build artifacts invalidated along contract boundaries, and AI repair is driven by proof obligations rather than prompts. A working verifier for Python, with contracts written as ordinary assertions, shows the model is implementable today. The open question we find most interesting is the one the model poses by construction: whether software development in the AI era is best understood as programming – or as specification management.

References

- [1] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [2] E. W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975.
- [3] J. C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proc. 17th IEEE Symposium on Logic in Computer Science (LICS)*, pages 55–74, 2002.
- [4] R. Jung, R. Krebbers, J.-H. Jourdan, A. Bizjak, L. Birkedal, and D. Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.
- [5] K. R. M. Leino. Dafny: An automatic program verifier for functional correctness. In *Proc. 16th Int. Conf. on Logic for Programming, Artificial Intelligence and Reasoning (LPAR-16)*, pages 348–370, 2010.
- [6] N. Swamy, C. Hrițcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, et al. Dependent types and multi-monadic effects in F*. In *Proc. 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 256–270, 2016.

- [7] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel. Verus: Verifying Rust programs using linear ghost types. *Proc. ACM on Programming Languages*, 7(OOPSLA1), 2023.
- [8] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.